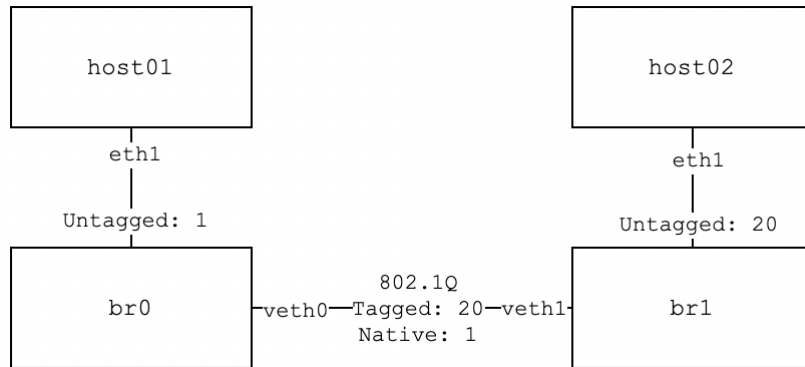


VLAN hopping [50 points]

In this assignment, we work with the topology shown in the image below. The two hosts are in different VLANs — `host01` (192.168.120.100) is in VLAN 1 and `host02` (192.168.130.100) is in VLAN 20. Under normal conditions, this prevents them from communicating with each other. ID 1 is the native VLAN.



However, such a setup is vulnerable to a double tag attack, allowing `host01` to send packets to `host02`.

We recommend running this assignment on Linux (or in a VM). To set up the environment, extract files from the provided ZIP and execute the setup script.

```
sudo ./setup_environment.sh
```

The two hosts are configured as Docker containers. As before, the *attacker* container (`host01` in the diagram above) has a mount configured; files in the `solution` directory on your computer can be accessed in the container under `/solution`. You can enter the container by executing the command below.

```
docker exec -it assignment_2_vlan_attacker bash
```

You can use `tcpdump` to view packets received by the other host. This can be done as follows:

```
docker exec -it assignment_2_vlan_host2 tcpdump -vv -n
```

For debugging, you might also find it useful to attach Wireshark or use `tcpdump` on various interfaces in the network in order to observe packets. Some such interfaces are `vlan_veth0` (connecting `host01` to `br0`), `vlan_trunk0` (connecting `br0` to `br1`), and `vlan_veth1` (connecting `host02` to `br1`).

After finishing the assignment, make sure to clean up the environment using the command below.

```
sudo ./destroy_environment.sh
```

Goal

Make a Python3 script that runs on the *attacker* container. The program should accept three parameters: the VLAN ID of the attacker host, the VLAN ID of the target host, and the IP address of the target. Execute the attack and send an ICMP packet (echo request / ping) that bypasses the VLAN restrictions and is correctly received and accepted by the target host.

```
python3 double_tag.py <first VLAN ID> <second VLAN ID> <destination IP>
```

You can verify whether the double tag attack succeeded by inspecting incoming traffic on *host02*:

```
$ docker exec -it assignment_2_vlan_host2 tcpdump -vv -n
[...]
12:23:51.130009 IP (tos 0x0, ttl 64, id 1, offset 0, flags [none], proto ICMP (1),
length 28)
    192.168.120.100 > 192.168.130.100: ICMP echo request, id 0, seq 0, length 8
12:23:51.130031 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.130.1
tell 192.168.130.100, length 28
12:23:52.151512 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.130.1
tell 192.168.130.100, length 28
12:23:53.175393 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 192.168.130.1
tell 192.168.130.100, length 28
[...]
```

You may not use external libraries or tools different from those installed on the provided *attacker* container.

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.